
Prepared By: Avanthika R

Course: Certified Data Analyst

Batch: 13 – April

Tools Used: SQL

Project: Medical Data History

Medical Data History

QUESTION 1:

Show first name, last name, and gender of patients who's gender is 'M'

QUERY:

```
select first_name, last_name, gender
```

```
from patients
```

```
where gender = "M";
```

Execution Screenshot:

The screenshot shows a SQL IDE interface with a query editor and a result grid. The query editor contains the following SQL query:

```
-- 1. Show first name, last name, and gender of patients who's gender is 'M'
select first_name, last_name, gender from patients where gender = "M";
```

The result grid displays the following data:

	first_name	last_name	gender
▶	Donald	Waterfield	M
	Mickey	Baasha	M
	Jiji	Sharma	M
	Blair	Diaz	M
	Charles	Wolfe	M
	Thomas	ONeill	M
	Sonny	Beckett	M
	Cedric	Coltrane	M
	Hank	Spencer	M
	Rick	Bennett	M
	Woody	Bashir	M

The interface also includes a toolbar with various icons, a 'Limit to 1000 rows' dropdown, and a 'Result Grid' button on the right side. The bottom status bar shows 'Vertical Output' and 'patients 1 x'.

QUESTION 2:

Show first name and last name of patients who does not have allergies

QUERY:

```
select first_name, last_name
from patients
where allergies = '';
```

Execution Screenshot:

The screenshot shows a SQL IDE interface with a query editor and a results grid. The query editor contains the following SQL query:

```
-- 2. Show first name and last name of patients who does not have allergies
select first_name, last_name from patients where allergies = '';
```

The results grid displays the following data:

first_name	last_name
Donald	Waterfield
Blair	Diaz
Thomas	O'Neill
Sonny	Beckett
Cedric	Coltrane
Hank	Spencer
Sara	di Marco
Amy	Leela
Rachel	Winterbourne
John	West
Jon	Doggett
Angel	Edwards
Brodie	Beck
Nea	Kobayakawa
Leona	Malik

The interface includes a toolbar with various icons, a 'Limit to 1000 rows' dropdown, and a 'Read Only' status indicator at the bottom right.

QUESTION 3:

Show first name of patients that start with the letter 'C'

QUERY:

```
select first_name  
from patients  
where first_name like 'C%';
```

Execution Screenshot:

The screenshot shows a SQL IDE interface with a query editor and a results pane. The query editor contains the following SQL query:

```
-- 3. Show first name of patients that start with the letter 'C'  
  
select first_name from patients where first_name like 'C%';
```

The results pane displays the output of the query, showing a list of first names starting with 'C'. The results are as follows:

first_name
Charles
Cedric
Charles
Cross
Calleigh
Catherine
Caroline
Casanova
Chen
Charmian
Cary
Christine
Carol
Corrado
Cordeiro

The interface also includes a toolbar with various icons, a 'Limit to 1000 rows' dropdown, and a 'Result Grid' button. The bottom status bar shows 'Vertical Output' and 'patients 3 x'.

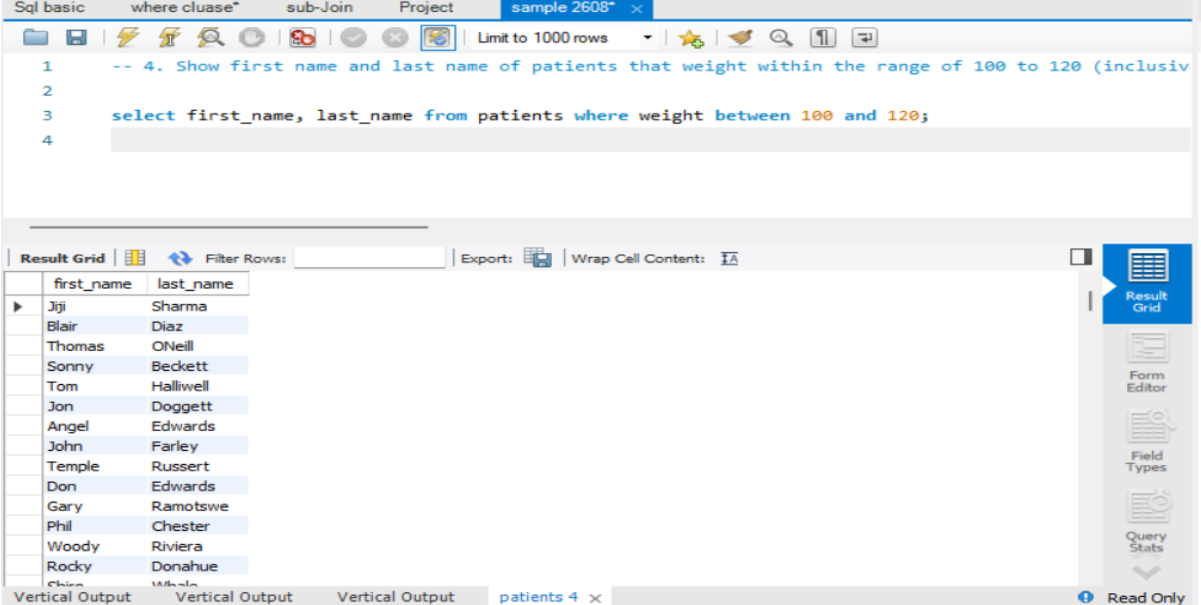
QUESTION 4:

Show first name and last name of patients that weight within the range of 100 to 120 (inclusive).

QUERY:

```
select first_name, last_name
from patients
where weight between 100 and 120;
```

Execution Screenshot:



The screenshot shows a SQL IDE window with a query editor and a result grid. The query editor contains the following SQL code:

```
-- 4. Show first name and last name of patients that weight within the range of 100 to 120 (inclusiv
select first_name, last_name from patients where weight between 100 and 120;
```

The result grid displays the following data:

first_name	last_name
Jiji	Sharma
Blair	Diaz
Thomas	O'Neill
Sonny	Beckett
Tom	Halliwel
Jon	Doggett
Angel	Edwards
John	Farley
Temple	Russert
Don	Edwards
Gary	Ramotswe
Phil	Chester
Woody	Riviera
Rocky	Donahue
Shirley	Wheeler

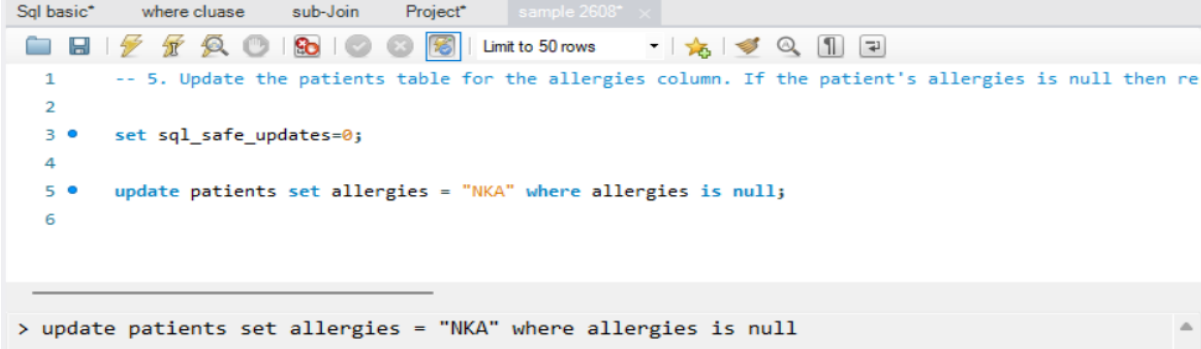
QUESTION 5:

Update the patients table for the allergies column. If the patient's allergies is null then replace it with 'NKA'.

QUERY:

```
update patients set allergies = "NKA" where allergies is null ;
```

EXECUTION SCREENSHOT:



The screenshot shows a SQL IDE window with a query editor. The query editor contains the following SQL code:

```
-- 5. Update the patients table for the allergies column. If the patient's allergies is null then re
set sql_safe_updates=0;
update patients set allergies = "NKA" where allergies is null;
```

The execution bar at the bottom shows the command: `> update patients set allergies = "NKA" where allergies is null`

QUESTION 6:

Show first name and last name concatenated into one column to show their full name.

QUERY:

```
select concat (first_name,' ',last_name) as full_name
from patients;
```

EXECUTION SCREENSHOT:

The screenshot shows a SQL IDE interface with a query editor and a result grid. The query editor contains the following SQL code:

```
-- 6. Show first name and last name concatenated into one column to show their full name.
select concat (first_name,' ',last_name) as full_name from patients;
```

The result grid displays the output of the query, showing a single column named 'full_name' with 15 rows of patient names concatenated. The names are: Donald Waterfield, Mickey Baasha, Jiji Sharma, Blair Diaz, Charles Wolfe, Sue Falcon, Thomas O'Neill, Sonny Beckett, Sister Spitzer, Cedric Coltrane, Hank Spencer, Sara di Marco, Daphne Seabright, Rick Bennett, and Amy Leake.

full_name
Donald Waterfield
Mickey Baasha
Jiji Sharma
Blair Diaz
Charles Wolfe
Sue Falcon
Thomas O'Neill
Sonny Beckett
Sister Spitzer
Cedric Coltrane
Hank Spencer
Sara di Marco
Daphne Seabright
Rick Bennett
Amy Leake

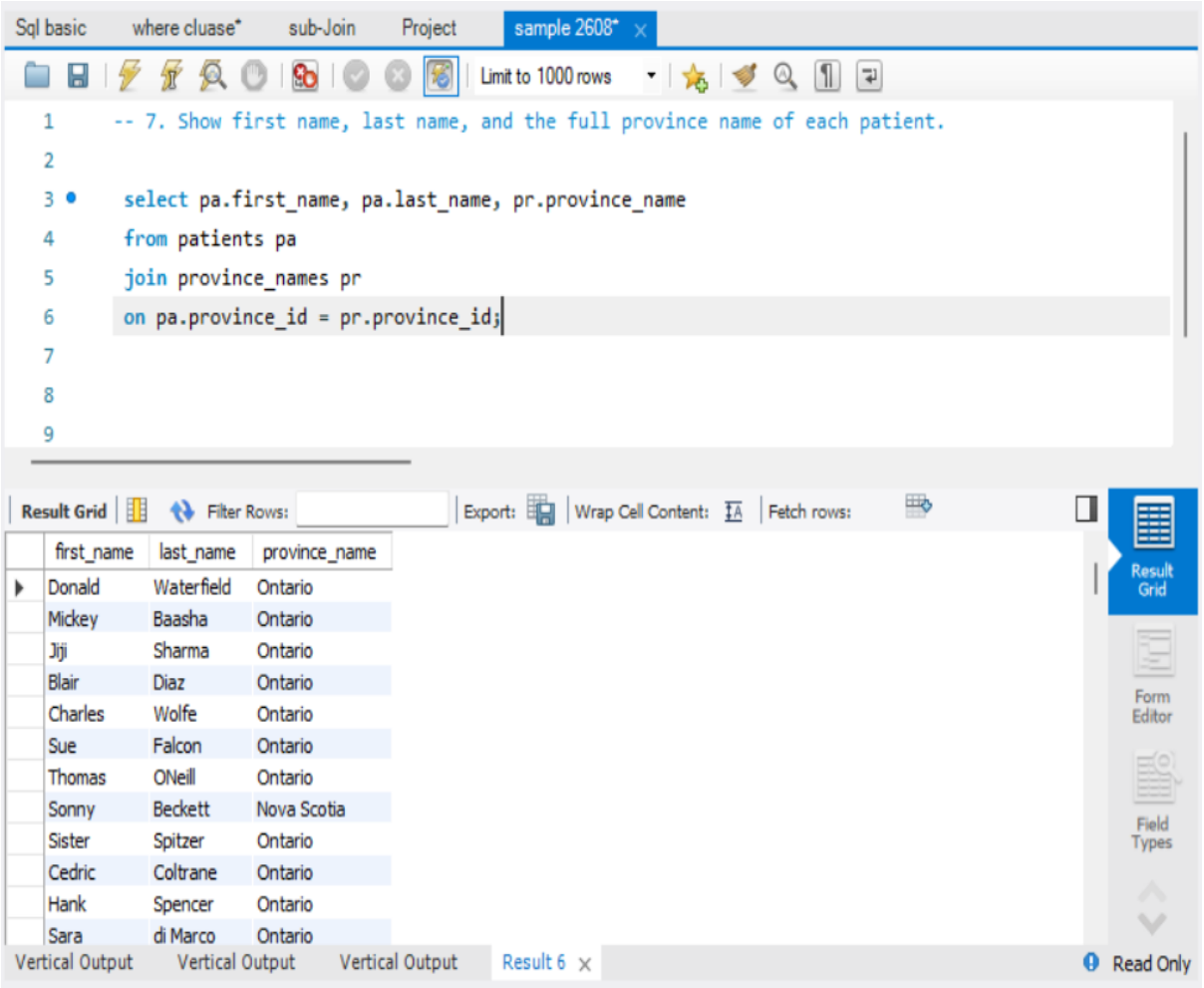
QUESTION 7:

Show first name, last name, and the full province name of each patient.

QUERY:

```
select pa.first_name, pa.last_name, pr.province_name
from patients as pa
join province_names as pr
on pa.province_id = pr.province_id;
```

EXECUTION SCREENSHOT:



The screenshot displays a SQL IDE interface with a query editor and a result grid. The query editor shows the following SQL query:

```
-- 7. Show first name, last name, and the full province name of each patient.

select pa.first_name, pa.last_name, pr.province_name
from patients pa
join province_names pr
on pa.province_id = pr.province_id;
```

The result grid shows the following data:

	first_name	last_name	province_name
▶	Donald	Waterfield	Ontario
	Mickey	Baasha	Ontario
	Jiji	Sharma	Ontario
	Blair	Diaz	Ontario
	Charles	Wolfe	Ontario
	Sue	Falcon	Ontario
	Thomas	O'Neill	Ontario
	Sonny	Beckett	Nova Scotia
	Sister	Spitzer	Ontario
	Cedric	Coltrane	Ontario
	Hank	Spencer	Ontario
	Sara	di Marco	Ontario

The interface includes a toolbar with icons for file operations, a 'Limit to 1000 rows' dropdown, and a 'Result Grid' button. The bottom status bar shows 'Vertical Output' and 'Result 6 x'.

QUESTION 8:

Show how many patients have a birth_date with 2010 as the birth year.

QUERY:

```
Select count(*) as Total_patients from patients
where year(str_to_date(birth_date,'%d-%m-%Y'))=2010;
```

EXECUTION SCREENSHOT:

The screenshot shows a SQL IDE window with a query editor and a result grid. The query editor contains the following SQL code:

```
-- 8. Show how many patients have a birth_date with 2010 as the birth year.
select count(*) as Total_patients from patients
where year(str_to_date(birth_date,'%d-%m-%Y'))=2010;
```

The result grid shows the following data:

Total_patients
55

QUESTION 9:

Show the first_name, last_name, and height of the patient with the greatest height.

QUERY:

```
select first_name, last_name, height
from patients
order by height desc limit 1;
```

EXECUTION SCREENSHOT:

The screenshot shows a SQL IDE window with a query editor and a result grid. The query editor contains the following SQL code:

```
-- 9. Show the first_name, last_name, and height of the patient with the greatest height.
select first_name, last_name, height from patients order by height desc limit 1;
```

The result grid shows the following data:

first_name	last_name	height
Sam	Haruko	226

QUESTION 10:

Show all columns for patients who have one of the following patient_ids: 1,45,534,879,1000.

QUERY:

```
select* from patients where patient_id in (1,45,534,879,1000);
```

EXECUTION SCREENSHOT:

The screenshot shows a SQL IDE window titled "sample 2608". The query editor contains the following SQL code:

```
-- 10. Show all columns for patients who have one of the following patient_ids: 1,45,534,879,1000
select* from patients where patient_id in (1,45,534,879,1000);
```

The "Result Grid" tab is active, displaying the following data:

patient_id	first_name	last_name	gender	birth_date	city	province_id	allergies	height	weight
1	Donald	Waterfield	M	12-02-1963	Barrie	ON		156	65
45	Cross	Gordon	M	20-03-2009	Ancaster	ON		125	53
534	Don	Zatara	M	11-01-2008	Timmins	ON		136	67
879	Orla	Shawn	F	24-09-1967	Sarnia	ON	Penicillin	149	65
1000	Rick	Williams	M	13-04-1975	Hamilton	ON	Penicillin	176	127

QUESTION 11:

Show the total number of admissions.

QUERY:

```
select count(*) as Total_Admission
from admissions;
```

EXECUTION SCREENSHOT:

The screenshot shows a SQL IDE window titled "sample 2608". The query editor contains the following SQL code:

```
-- 11. Show the total number of admissions
select count(*) as Total_Admission from admissions;
```

The "Result Grid" tab is active, displaying the following data:

Total_Admission
4915

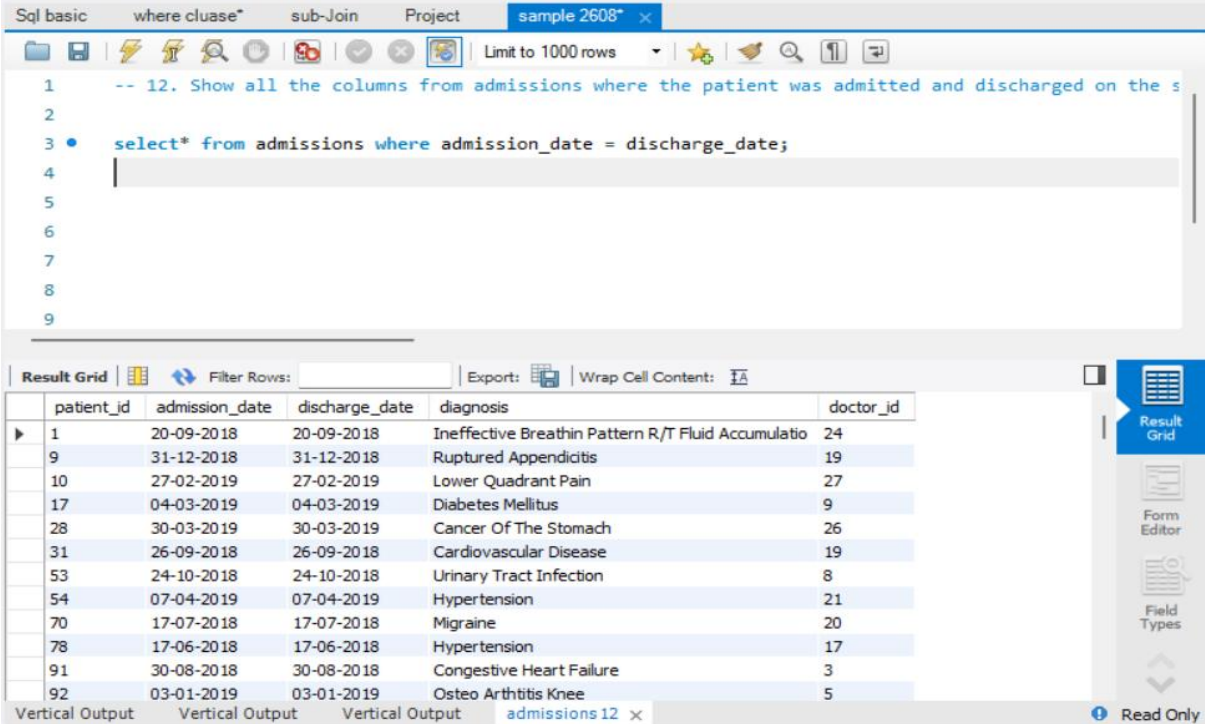
QUESTION 12:

Show all the columns from admissions where the patient was admitted and discharged on the same day.

QUERY:

```
select* from admissions where admission_date = discharge_date;
```

EXECUTION SCREENSHOT:



The screenshot shows a SQL IDE interface with a query editor and a result grid. The query editor contains the following SQL statement:

```
-- 12. Show all the columns from admissions where the patient was admitted and discharged on the same day.
select* from admissions where admission_date = discharge_date;
```

The result grid displays the following data:

patient_id	admission_date	discharge_date	diagnosis	doctor_id
1	20-09-2018	20-09-2018	Ineffective Breathing Pattern R/T Fluid Accumulation	24
9	31-12-2018	31-12-2018	Ruptured Appendicitis	19
10	27-02-2019	27-02-2019	Lower Quadrant Pain	27
17	04-03-2019	04-03-2019	Diabetes Mellitus	9
28	30-03-2019	30-03-2019	Cancer Of The Stomach	26
31	26-09-2018	26-09-2018	Cardiovascular Disease	19
53	24-10-2018	24-10-2018	Urinary Tract Infection	8
54	07-04-2019	07-04-2019	Hypertension	21
70	17-07-2018	17-07-2018	Migraine	20
78	17-06-2018	17-06-2018	Hypertension	17
91	30-08-2018	30-08-2018	Congestive Heart Failure	3
92	03-01-2019	03-01-2019	Osteo Arthritis Knee	5

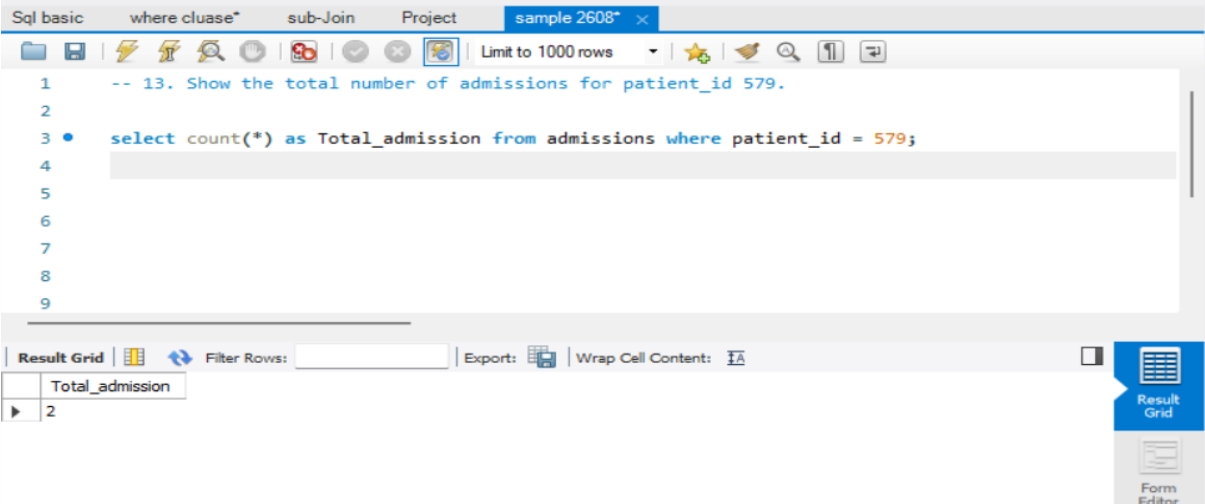
QUESTION 13:

Show the total number of admissions for patient_id 579.

QUERY:

```
select count(*) as Total_admission from admissions where patient_id = 579;
```

EXECUTION SCREENSHOT:



The screenshot shows a SQL IDE interface with a query editor and a result grid. The query editor contains the following SQL statement:

```
-- 13. Show the total number of admissions for patient_id 579.
select count(*) as Total_admission from admissions where patient_id = 579;
```

The result grid displays the following data:

Total_admission
2

QUESTION 14:

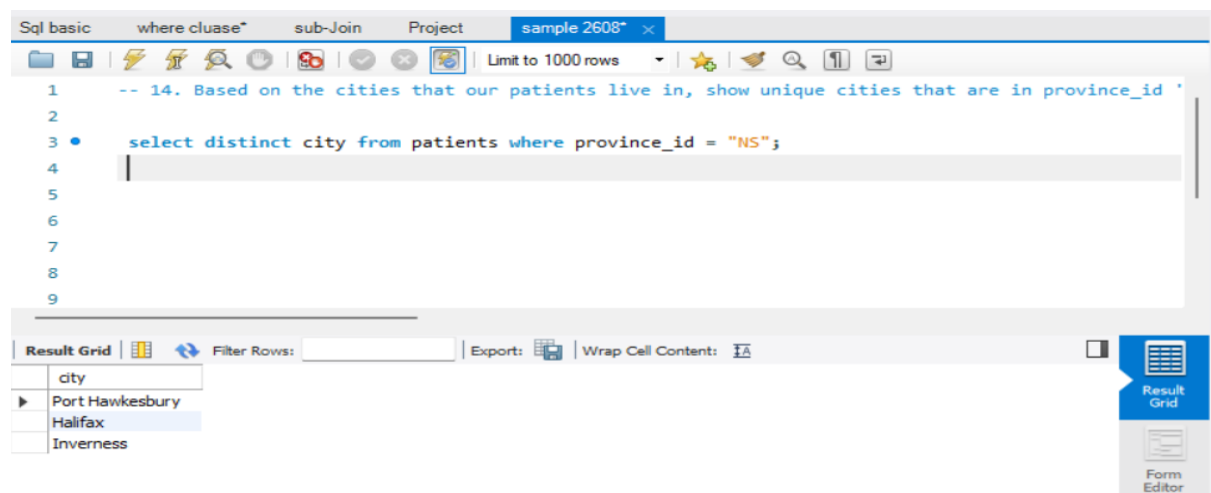
Based on the cities that our patients live in, show unique cities that are in province_id 'NS'?

QUERY:

```
select distinct city from patients
```

```
where province_id = "NS";
```

EXECUTION SCREENSHOT:



QUESTION 15:

Write a query to find the first_name, last name and birth date of patients who have height more than 160 and weight more than 70.

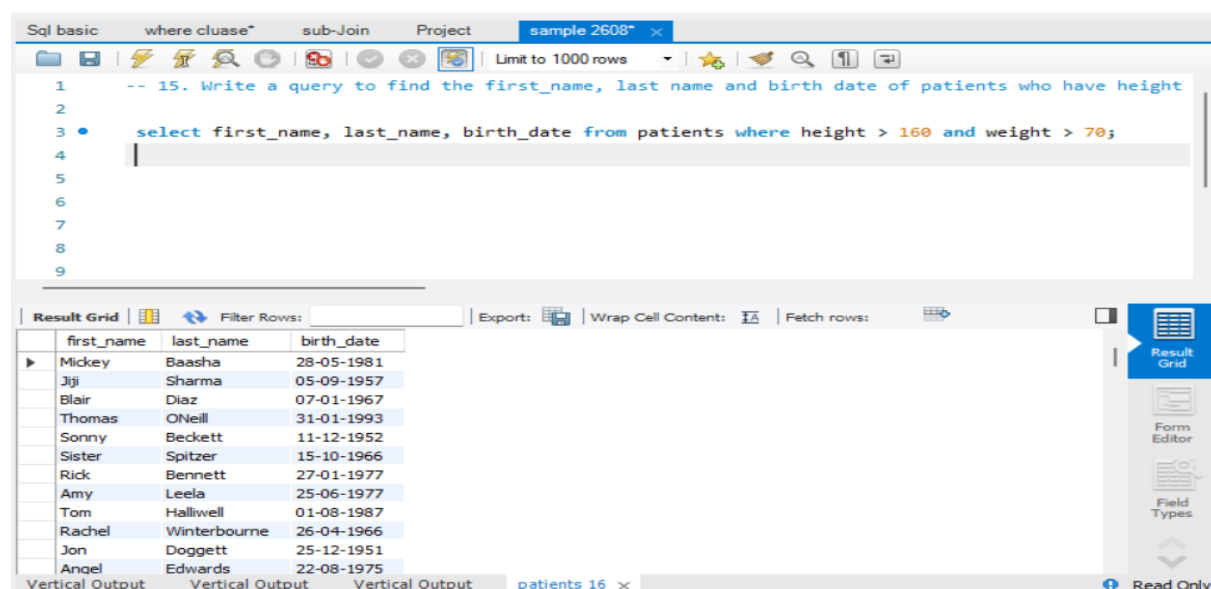
QUERY:

```
select first_name, last_name, birth_date
```

```
from patients
```

```
where height > 160 and weight > 70;
```

EXECUTION SCREENSHOT:



QUESTION 16:

Show unique birth years from patients and order them by ascending.

QUERY:

```
select distinct year(str_to_date(birth_date,'%d-%m-%Y')) as Birth_Year
from patients
order by year(str_to_date(birth_date,'%d-%m-%Y'));
```

EXECUTION SCREENSHOT:

The screenshot shows a SQL IDE interface with a query editor and a result grid. The query editor contains the following SQL code:

```
-- 16. Show unique birth years from patients and order them by ascending.
select distinct year(str_to_date(birth_date,'%d-%m-%Y')) as Birth_Year
from patients order by year(str_to_date(birth_date,'%d-%m-%Y'));
```

The result grid displays the output of the query, showing a list of unique birth years in ascending order:

Birth_Year
1918
1923
1925
1926
1927
1928
1929
1931
1933
1934
1936
1937

The IDE interface includes a toolbar with various icons for file operations, a 'Limit to 1000 rows' dropdown, and a sidebar with options for 'Result Grid', 'Form Editor', and 'Field Types'. The bottom status bar shows 'Vertical Output' and 'Result 17 x'.

QUESTION 17:

Show unique first names from the patients table which only occurs once in the list.

QUERY:

```
select first_name
from patients
group by first_name
having count(first_name) =1;
```

EXECUTION SCREENSHOT:

The screenshot shows a SQL IDE interface with a query editor and a results pane. The query editor contains the following SQL code:

```
-- 17. Show unique first names from the patients table which only occurs once in the list.
select first_name from patients
group by first_name having count(first_name) =1;
```

The results pane displays the output of the query, showing a list of unique first names from the patients table. The names are listed in a table with a single column labeled 'first_name'.

first_name
Angel
Brodie
Anne
Gytha
Zoe
Jonny
Priestley
Mattie
Vin
Robbie
Valentin
Nina

The interface also shows a sidebar with options like 'Result Grid', 'Form Editor', and 'Field Types'. The bottom status bar indicates 'patients 18 x' and 'Read Only'.

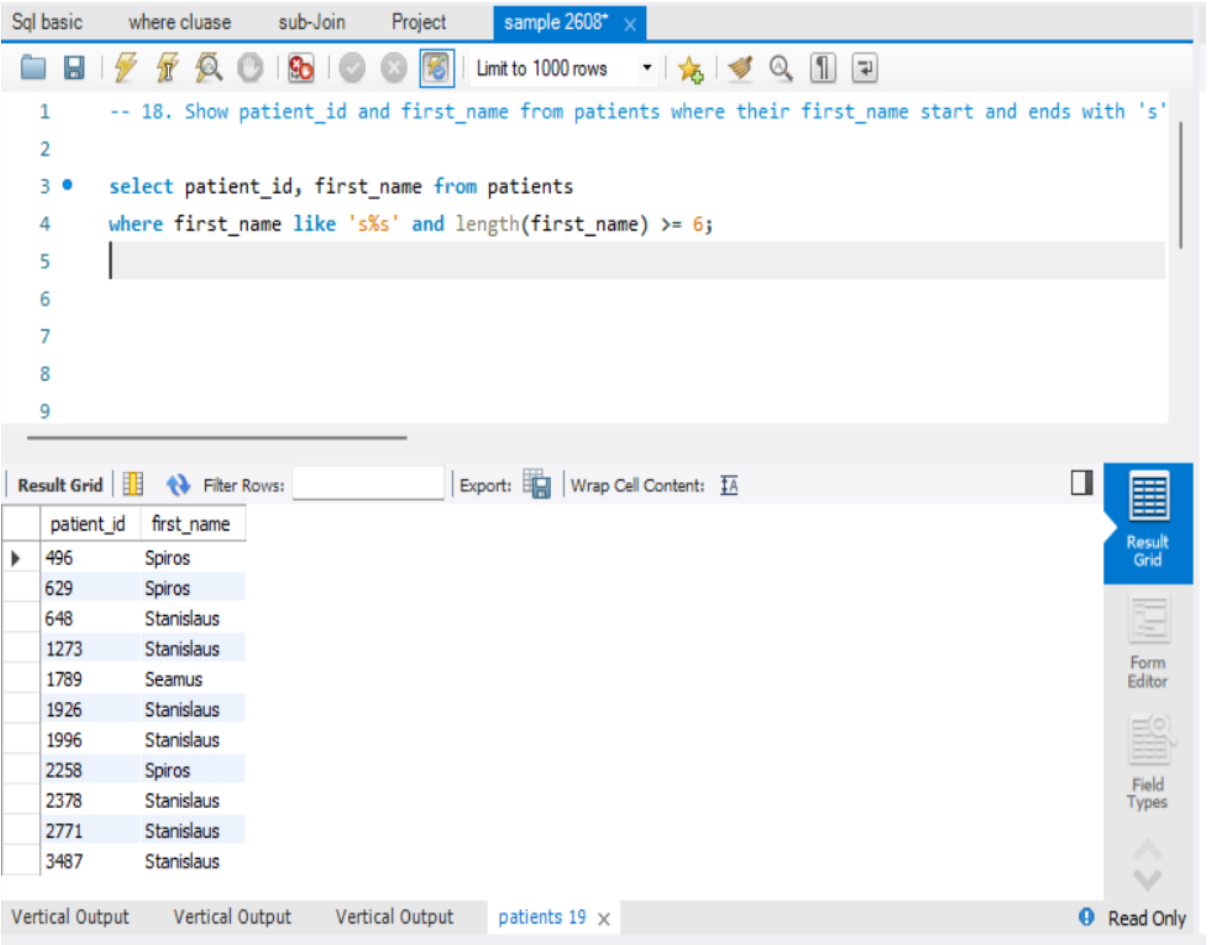
QUESTION 18:

Show patient_id and first_name from patients where their first_name start and ends with 's' and is at least 6 characters long.

QUERY:

```
select patient_id, first_name
from patients
where first_name like 's%s' and length(first_name) >= 6;
```

EXECUTION SCREENSHOT:



The screenshot shows a SQL IDE interface with a query editor and a result grid. The query editor contains the following SQL code:

```
-- 18. Show patient_id and first_name from patients where their first_name start and ends with 's'
select patient_id, first_name from patients
where first_name like 's%s' and length(first_name) >= 6;
```

The result grid displays the following data:

patient_id	first_name
496	Spiros
629	Spiros
648	Stanislaus
1273	Stanislaus
1789	Seamus
1926	Stanislaus
1996	Stanislaus
2258	Spiros
2378	Stanislaus
2771	Stanislaus
3487	Stanislaus

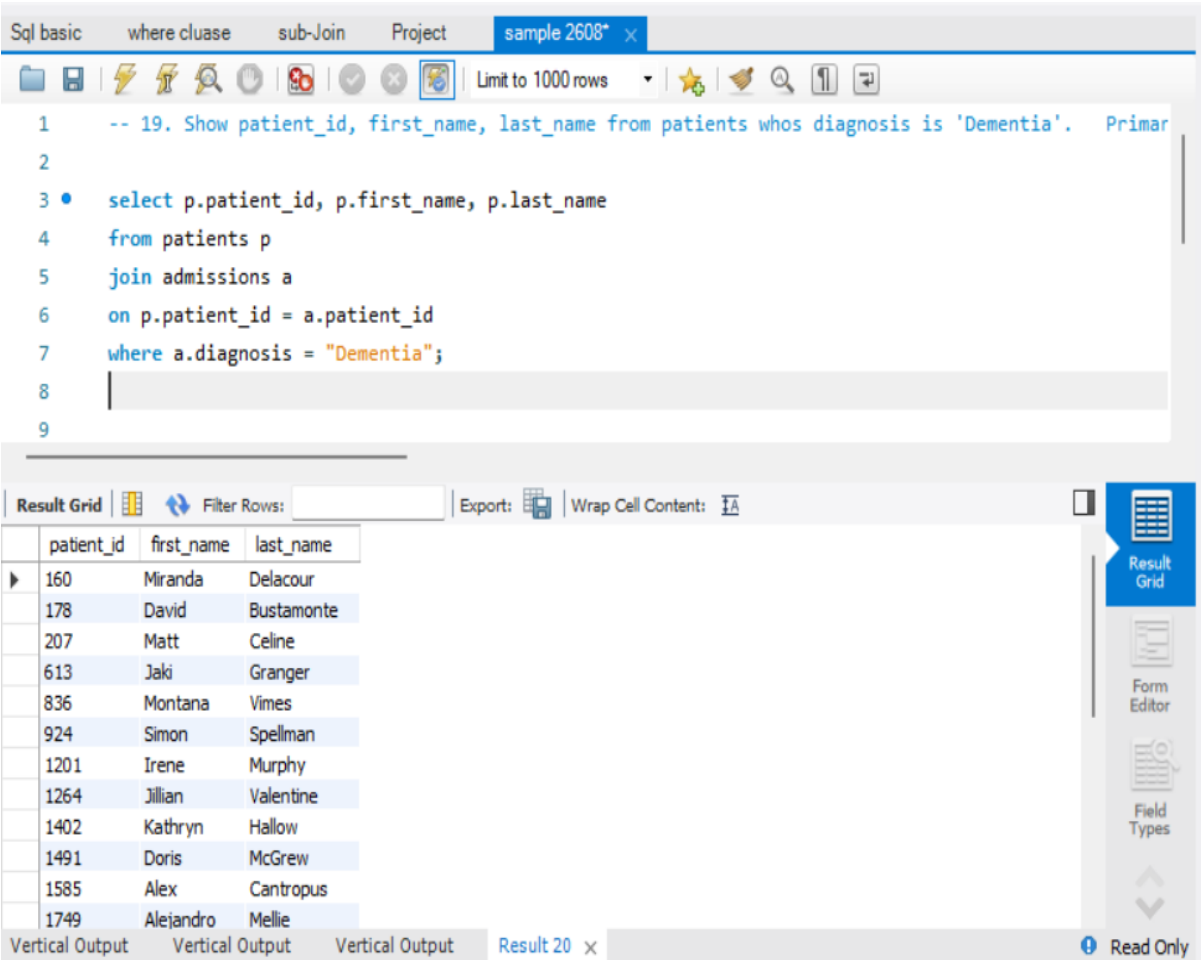
QUESTION 19:

Show patient_id, first_name, last_name from patients whos diagnosis is 'Dementia'. Primary diagnosis is stored in the admissions table.

QUERY:

```
select p.patient_id, p.first_name, p.last_name
from patients p
join admissions a
on p.patient_id = a.patient_id
where a.diagnosis = "Dementia";
```

EXECUTION SCREENSHOT:



The screenshot displays a SQL IDE interface. The top toolbar includes icons for file operations, execution, and a 'Limit to 1000 rows' dropdown. The SQL editor contains the following query:

```
-- 19. Show patient_id, first_name, last_name from patients whos diagnosis is 'Dementia'.  Primar
2
3 • select p.patient_id, p.first_name, p.last_name
4 from patients p
5 join admissions a
6 on p.patient_id = a.patient_id
7 where a.diagnosis = "Dementia";
8
9
```

Below the editor, the 'Result Grid' tab is active, showing the query results in a table with 13 rows and 3 columns: patient_id, first_name, and last_name.

patient_id	first_name	last_name
160	Miranda	Delacour
178	David	Bustamonte
207	Matt	Celine
613	Jaki	Granger
836	Montana	Vimes
924	Simon	Spellman
1201	Irene	Murphy
1264	Jillian	Valentine
1402	Kathryn	Hallow
1491	Doris	McGrew
1585	Alex	Cantropus
1749	Alejandro	Mellie

The bottom status bar indicates 'Result 20 x' and 'Read Only'.

QUESTION 20:

Display every patient's first_name. Order the list by the length of each name and then by alphabetically.

QUERY:

```
select first_name
from patients
order by length(first_name) asc , first_name asc;
```

EXECUTION SCREENSHOT:

The screenshot shows a SQL IDE interface with a query editor and a results grid. The query editor contains the following SQL code:

```
-- 20. Display every patient's first_name. Order the list by the length of each name and then by a
select first_name from patients order by length(first_name) asc , first_name asc;
```

The results grid displays the first_name column for the patients. The results are ordered by the length of the first_name (ascending) and then alphabetically. The results are as follows:

first_name
Al
Al
Al
Al
Al
Al
Bo
Bo
Bo
Bo
Bo

The IDE interface includes a toolbar with various icons, a status bar at the bottom, and a sidebar on the right with options like Result Grid, Form Editor, and Field Types.

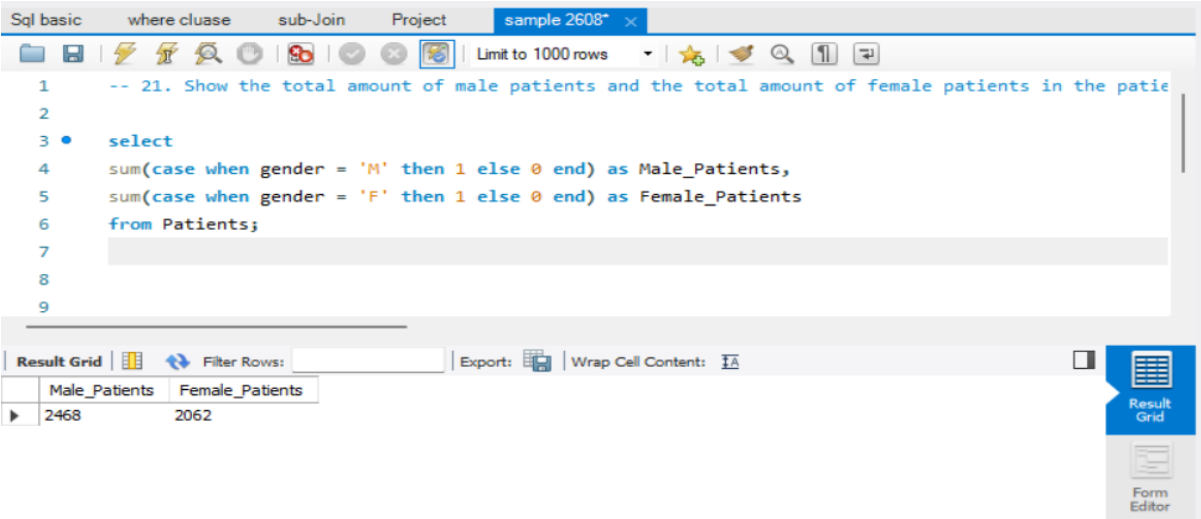
QUESTION 21:

Show the total amount of male patients and the total amount of female patients in the patients table. Display the two results in the same row.

QUERY:

```
select
sum(case when gender = 'M' then 1 else 0 end) as Male_Patients,
sum(case when gender = 'F' then 1 else 0 end) as Female_Patients
from Patients;
```

EXECUTION SCREENSHOT:



The screenshot shows a SQL IDE interface with a query editor and a result grid. The query is as follows:

```
-- 21. Show the total amount of male patients and the total amount of female patients in the patie
select
sum(case when gender = 'M' then 1 else 0 end) as Male_Patients,
sum(case when gender = 'F' then 1 else 0 end) as Female_Patients
from Patients;
```

The result grid displays the following data:

	Male_Patients	Female_Patients
▶	2468	2062

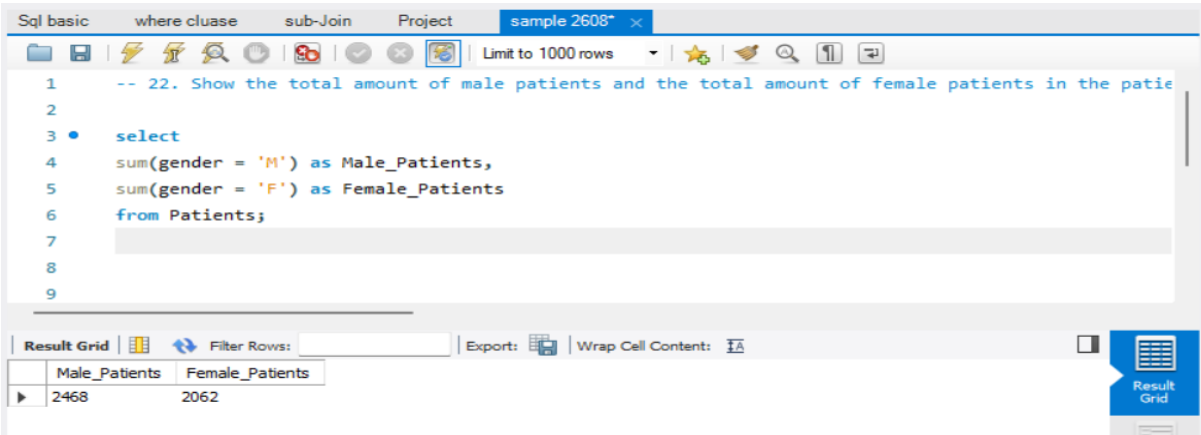
QUESTION 22:

Show the total amount of male patients and the total amount of female patients in the patients table. Display the two results in the same row.

QUERY:

```
Select sum(gender='M') as Male_Patients,
sum(gender='F') as Female_Patients from Patients;
```

EXECUTION SCREENSHOT:



The screenshot shows a SQL IDE interface with a query editor and a result grid. The query is as follows:

```
-- 22. Show the total amount of male patients and the total amount of female patients in the patie
select
sum(gender = 'M') as Male_Patients,
sum(gender = 'F') as Female_Patients
from Patients;
```

The result grid displays the following data:

	Male_Patients	Female_Patients
▶	2468	2062

QUESTION 23:

Show patient_id, diagnosis from admissions. Find patients admitted multiple times for the same diagnosis.

QUERY:

```
select patient_id,diagnosis
from admissions
group by patient_id,diagnosis
having count(*)>1;
```

EXECUTION SCREENSHOT:

The screenshot shows a SQL IDE interface. The top toolbar includes icons for file operations, execution, and a 'Limit to 1000 rows' dropdown. The SQL editor contains the following query:

```
-- 23. Show patient_id, diagnosis from admissions. Find patients admitted multiple times for the s
2
3 • select patient_id,diagnosis from admissions
4 group by patient_id,diagnosis having count(*)>1;
5
6
7
8
9
```

Below the editor is the 'Result Grid' section, which includes a 'Filter Rows' input, an 'Export' button, and a 'Wrap Cell Content' checkbox. The results are displayed in a table:

patient_id	diagnosis
137	Pregnancy
320	Pneumonia
1577	Congestive Heart Failure
2004	Left Shoulder Rotator Cuff Repair
2859	Severed Spine At C3
3012	Appendicitis
3367	Pyelonephritis
3468	Congestive Heart Failure
4083	Congestive Heart Failure
4121	Congestive Heart Failure
4363	Congestive Heart Failure

On the right side of the interface, there are buttons for 'Result Grid', 'Form Editor', and 'Field Types'. The bottom status bar shows 'Vertical Output' tabs, a tab for 'admissions26', and a 'Read Only' indicator.

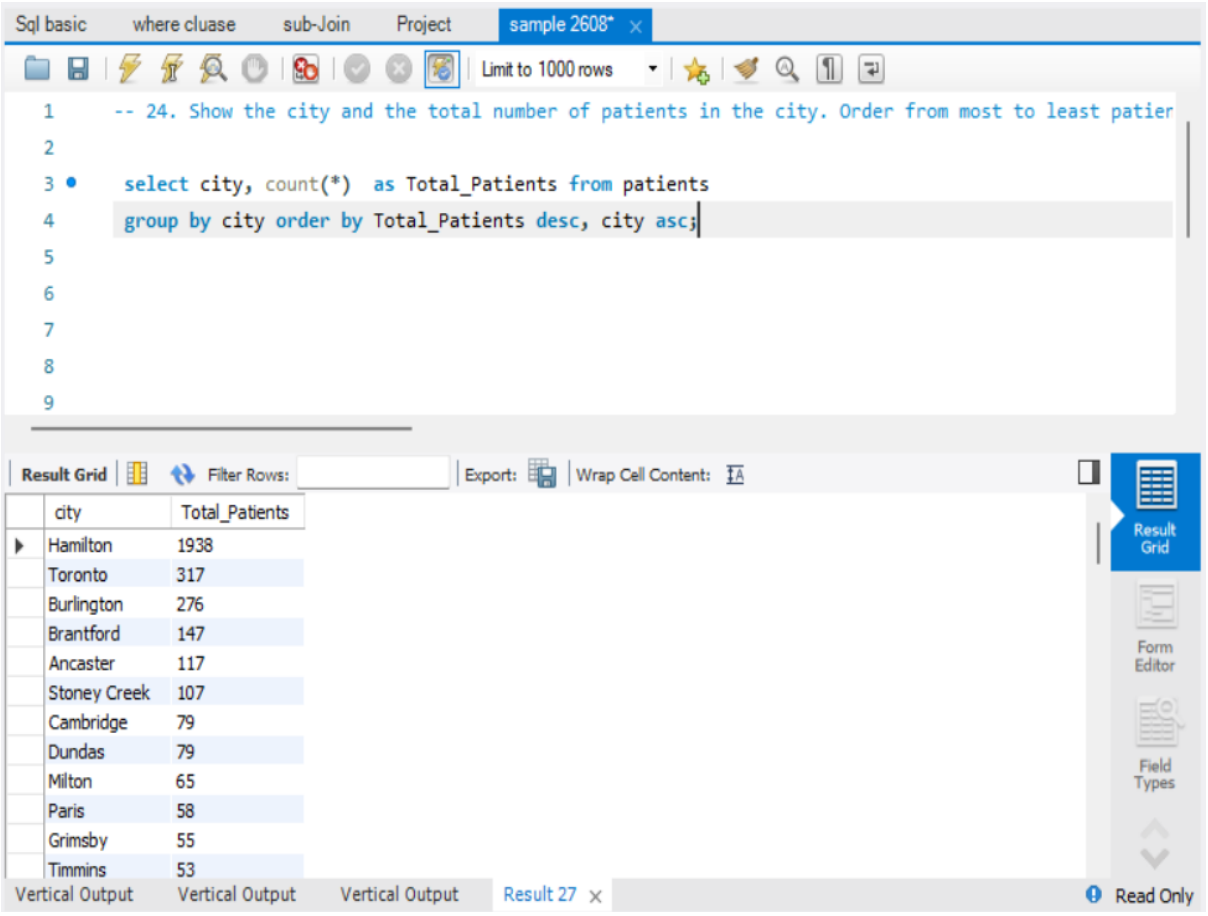
QUESTION 24:

Show the city and the total number of patients in the city. Order from most to least patients and then by city name ascending.

QUERY:

```
select city, count(*) as Total_Patients
from patients
group by city
order by Total_Patients desc, city asc;
```

EXECUTION SCREENSHOT:



The screenshot shows a SQL IDE interface with a query editor and a result grid. The query is: `select city, count(*) as Total_Patients from patients group by city order by Total_Patients desc, city asc;`. The result grid displays the following data:

city	Total_Patients
Hamilton	1938
Toronto	317
Burlington	276
Brantford	147
Ancaster	117
Stoney Creek	107
Cambridge	79
Dundas	79
Milton	65
Paris	58
Grimsby	55
Timmins	53

QUESTION 25:

Show first name, last name and role of every person that is either patient or doctor. The roles are either "Patient" or "Doctor".

QUERY:

```
select first_name, last_name, 'Patient' as Roles
from patients
union
select first_name, last_name, 'Doctor' as Roles
from doctors;
```

EXECUTION SCREENSHOT:

The screenshot shows a SQL IDE window titled "sample 2608". The query editor contains the following SQL code:

```
-- 25. Show first name, last name and role of every person that is either patient or doctor. The r
select first_name, last_name, 'Patient' as Roles
from patients
union
select first_name, last_name, 'Doctor' as Roles
from doctors;
```

The results are displayed in a table with the following columns: first_name, last_name, Roles. The table contains 14 rows of data, with the last row showing a Doctor.

first_name	last_name	Roles
Morgan	McCrary	Patient
Sam	OHara	Patient
Charlie	Neutron	Patient
Fleur	Morales	Patient
Faye	Soong	Patient
Christine	Turtle	Patient
Armando	Matthews	Patient
Karen	Costello	Patient
Deanna	Barnes	Patient
Molly	Hanover	Patient
Constance	Yang	Patient
Claude	Walls	Doctor

The IDE interface includes a toolbar with icons for file operations, a "Limit to 1000 rows" dropdown, and a "Result Grid" button. The bottom status bar shows "Vertical Output" and "Result 28 x".

QUESTION 26:

Show all allergies ordered by popularity. Remove NULL values from query.

QUERY:

```
select allergies, count(*) as Popularity
from patients
where allergies is not null and trim(allergies) != ''
group by allergies
order by Popularity desc;
```

EXECUTION SCREENSHOT:

The screenshot displays a SQL IDE interface. The top toolbar includes icons for file operations, execution, and a 'Limit to 1000 rows' dropdown. The SQL editor contains the following query:

```
-- 26. Show all allergies ordered by popularity. Remove NULL values from query.

select allergies, count(*) as Popularity from patients
where allergies is not null and trim(allergies) != ''
group by allergies
order by Popularity desc;
```

Below the editor, the 'Result Grid' tab is active, showing the query results in a table:

allergies	Popularity
Penicillin	1082
Codeine	305
Sulfa	157
ASA	99
Sulfa Drugs	71
Peanuts	52
Iodine	48
Tylenol	42
Bee Stings	40
Valporic Acid	38
Tetracycline	34
Wheat	33

The bottom of the IDE shows a 'Read Only' status and a 'Result 32' tab.

QUESTION 27:

Show all patient's first_name, last_name, and birth_date who were born in the 1970s decade. Sort the list starting from the earliest birth_date.

QUERY:

```
select first_name,last_name, birth_date
from patients
where year(str_to_date(birth_date, '%d-%m-%Y')) between 1970 and 1979
order by str_to_date(birth_date, '%d-%m-%Y');
```

EXECUTION SCREENSHOT:

The screenshot shows a SQL IDE interface with a query editor at the top and a result grid at the bottom. The query editor contains the following SQL code:

```
-- 27. Show all patient's first_name, last_name, and birth_date who were born in the 1970s decade. S
2
3 select first_name,last_name, birth_date from patients
4 where year(str_to_date(birth_date, '%d-%m-%Y')) between 1970 and 1979
5 order by str_to_date(birth_date, '%d-%m-%Y');
6
```

The result grid displays the following data:

	first_name	last_name	birth_date
▶	Frances	Kobayakawa	02-01-1970
	Sunny	Burrell	07-01-1970
	Penelope	Beckett	14-01-1970
	Deborah	Stewart	14-01-1970
	Augusta	Decker	22-01-1970
	Sookie	Brearily	01-02-1970
	Temple	Wylie	10-02-1970
	Deanna	Spano	23-03-1970
	Jadu	Principal	28-03-1970
	Betty	Stephens	28-03-1970
	Jo	Sahid	31-03-1970
	Santino	Hawker	08-04-1970

The interface also includes a toolbar with various icons, a "Limit to 1000 rows" dropdown, and a "Read Only" status indicator at the bottom right.

QUESTION 28:

We want to display each patient's full name in a single column. Their last_name in all upper letters must appear first, then first_name in all lower case letters. Separate the last_name and first_name with a comma. Order the list by the first_name in descending order EX: SMITH,jane.

QUERY:

```
select concat( upper(last_name),',', lower(first_name)) as Full_name
from patients
order by first_name desc;
```

EXECUTION SCREENSHOT:

The screenshot displays a SQL IDE interface. The top toolbar includes icons for file operations, execution, and a 'Limit to 50 rows' dropdown. The SQL editor contains the following query:

```
-- 28. We want to display each patient's full name in a single column. Their last_name in all upper
select concat( upper(last_name),',', lower(first_name)) as Full_name
from patients order by first_name desc;
```

The bottom section shows the 'Result Grid' with the following data:

Full_name
MILLER,zoe
CORBIE,ziva
KOBAYAKAWA,zenigata
OVERSTREET,zenigata
BENNETT,zen
MEPHESTO,zelda
MORRIS,zelda
THOMAS,zefram
FLUTE,zefram
MARONEY,zatanna
TYRELL,zatanna
CHE,zane

The bottom status bar indicates 'Vertical Output', 'Vertical Output', 'Vertical Output', 'Result 35', and 'Read Only'.

QUESTION 29:

Show the province_id(s), sum of height; where the total sum of its patient's height is greater than or equal to 7,000.

QUERY:

```
select province_id, sum(height) as Total_height
from patients
group by province_id having sum(height) >= 7000;
```

EXECUTION SCREENSHOT:

Sql basic | where clause | sub-Join | Project | sample 2608*

```
-- 29. Show the province_id(s), sum of height; where the total sum of its patient's height is greater than or equal to 7,000.

select province_id, sum(height) as Total_height from patients
group by province_id
having sum(height) >= 7000;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

province_id	Total_height
ON	678037
NS	9765
BC	7720

Result Grid | Form Editor

QUESTION 30:

Show the difference between the largest weight and smallest weight for patients with the last name 'Maroni'.

QUERY:

```
select max(weight)-min(weight) as Weight_difference from patients
where last_name = 'Maroni';
```

EXECUTION SCREENSHOT:

Sql basic | where clause | sub-Join | Project | sample 2608*

```
-- 30. Show the difference between the largest weight and smallest weight for patients with the last name 'Maroni'.

select max(weight)-min(weight) as Weight_difference
from patients where last_name = 'Maroni';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

Weight_difference
71

Result Grid

QUESTION 31:

Show all of the days of the month (1-31) and how many admission_dates occurred on that day. Sort by the day with most admissions to least admissions.

QUERY:

```
select day(str_to_date(admission_date,'%d-%m-%Y')) as Days,  
count(*) as Total_Admission  
from admissions  
group by day(str_to_date(admission_date,'%d-%m-%Y'))  
order by Total_admission desc;
```

EXECUTION SCREENSHOT:

The screenshot displays a SQL IDE interface. The top toolbar includes icons for file operations, execution, and a 'Limit to 50 rows' dropdown. The SQL editor contains the following query:

```
-- 31. Show all of the days of the month (1-31) and how many admission_dates occurred on that day. S  
select day(str_to_date(admission_date,'%d-%m-%Y')) as Days, count(*) as Total_Admission  
from admissions  
group by day(str_to_date(admission_date,'%d-%m-%Y'))  
order by Total_admission desc;
```

Below the editor, the 'Result Grid' tab is active, showing the query results in a table:

Days	Total_Admission
11	180
4	179
9	179
6	174
12	174
16	172
2	171
21	169
28	168
13	165
8	165
20	164

The right sidebar contains icons for 'Result Grid', 'Form Editor', and 'Field Types'. The bottom status bar shows 'Vertical Output' and 'Result 38'.

QUESTION 32:

Show all of the patients grouped into weight groups. Show the total amount of patients in each weight group. Order the list by the weight group descending. e.g. if they weight 100 to 109 they are placed in the 100 weight group, 110-119 = 110 weight group, etc.

QUERY:

```
select  
floor(weight / 10) * 10 as weight_group,  
count(*) as total_patients  
from patients  
group by floor(weight / 10) * 10  
order by weight_group desc;
```

EXECUTION SCREENSHOT:

The screenshot displays a SQL IDE interface. The top toolbar includes icons for file operations, execution, and a 'Limit to 50 rows' dropdown. The SQL editor contains the following query:

```
-- 32. Show all of the patients grouped into weight groups. Show the total amount of patients in each weight group.  
select  
  floor(weight / 10) * 10 as weight_group,  
  count(*) as total_patients  
from patients  
group by floor(weight / 10) * 10  
order by weight_group desc;
```

Below the editor, the 'Result Grid' tab is active, showing the query results in a table:

weight_group	total_patients
140	6
130	59
120	191
110	426
100	507
90	403
80	478
70	633
60	685
50	440

The bottom status bar indicates 'Result 6' and 'Read Only'.

QUESTION 33:

Show patient_id, weight, height, isObese from the patients table. Display isObese as a boolean 0 or 1. Obese is defined as $\text{weight(kg)} / (\text{height(m)})^2 \geq 30$. Weight is in units kg. Height is in units cm.

QUERY:

```
select patient_id, weight,height,
case
when weight/power(height/100.0,2) >= 30 then 1
else 0
end isobese
from patients;
```

EXECUTION SCREENSHOT:

The screenshot shows a SQL IDE interface with a query editor and a result grid. The query editor contains the following SQL code:

```
-- 33. Show patient_id, weight, height, isObese from the patients table. Display isObese as a bool
select patient_id, weight,height,
case
when weight/power(height/100.0,2) >= 30 then 1
else 0
end isobese
from patients;
```

The result grid displays the following data:

	patient_id	weight	height	isobese
1	1	65	156	0
2	2	76	185	0
3	3	106	194	0
4	4	104	191	0
5	5	10	47	1
6	6	5	43	0
7	7	117	180	1
8	8	105	174	1
9	9	95	173	1
10	10	61	157	0
11	11	74	158	0
12	12	46	145	0

The interface also includes a toolbar with various icons, a 'Limit to 50 rows' dropdown, and a 'Read Only' status indicator at the bottom right.

QUESTION 34:

Show patient_id, first_name, last_name, and attending doctor's specialty. Show only the patients who has a diagnosis as 'Epilepsy' and the doctor's first name is 'Lisa'. Check patients, admissions, and doctors tables for required information.

QUERY:

```
select p.patient_id, p.first_name, p.last_name,d.specialty
from patients p
join admissions a
on p.patient_id = a.patient_id
join doctors d
on a.doctor_id = d.doctor_id
where a.diagnosis = 'Epilepsy' and d.first_name = 'Lisa';
```

EXECUTION SCREENSHOT:

The screenshot shows a SQL IDE interface with a query editor and a result grid. The query editor contains the following SQL code:

```
-- 34. Show patient_id, first_name, last_name, and attending doctor's specialty. Show only the pat
3
4 • select p.patient_id, p.first_name, p.last_name,d.specialty
5   from patients p
6   join admissions a
7   on p.patient_id = a.patient_id
8   join doctors d
9   on a.doctor_id = d.doctor_id
10  where a.diagnosis = 'Epilepsy' and d.first_name = 'Lisa';
11
```

The result grid displays the following data:

patient_id	first_name	last_name	specialty
468	Frank	Anderson	Obstetrician/Gynecologist
701	Precious	Ashton	Obstetrician/Gynecologist